

Artificial Intelligence

AI2002

Rushda Muneer

Spring 2026

Adversarial Search

- In **Multiagent environments** each agent needs to consider the actions of other agents and how they affect its own welfare. The unpredictability of these other agents can introduce contingencies into the agent's problem-solving process.
- In **competitive environments**, the agents' goals are in conflict, giving rise to adversarial search problems—often used in games.
- In AI, the most common games are of a rather specialized kind—what game theorists call deterministic, turn-taking, two-player, zero-sum games of perfect information.
- In our terminology, this means deterministic, fully observable environments in which two agents act alternately and in which the **utility values** at the end of the game are always **equal** and **opposite**.

Game as a search problem

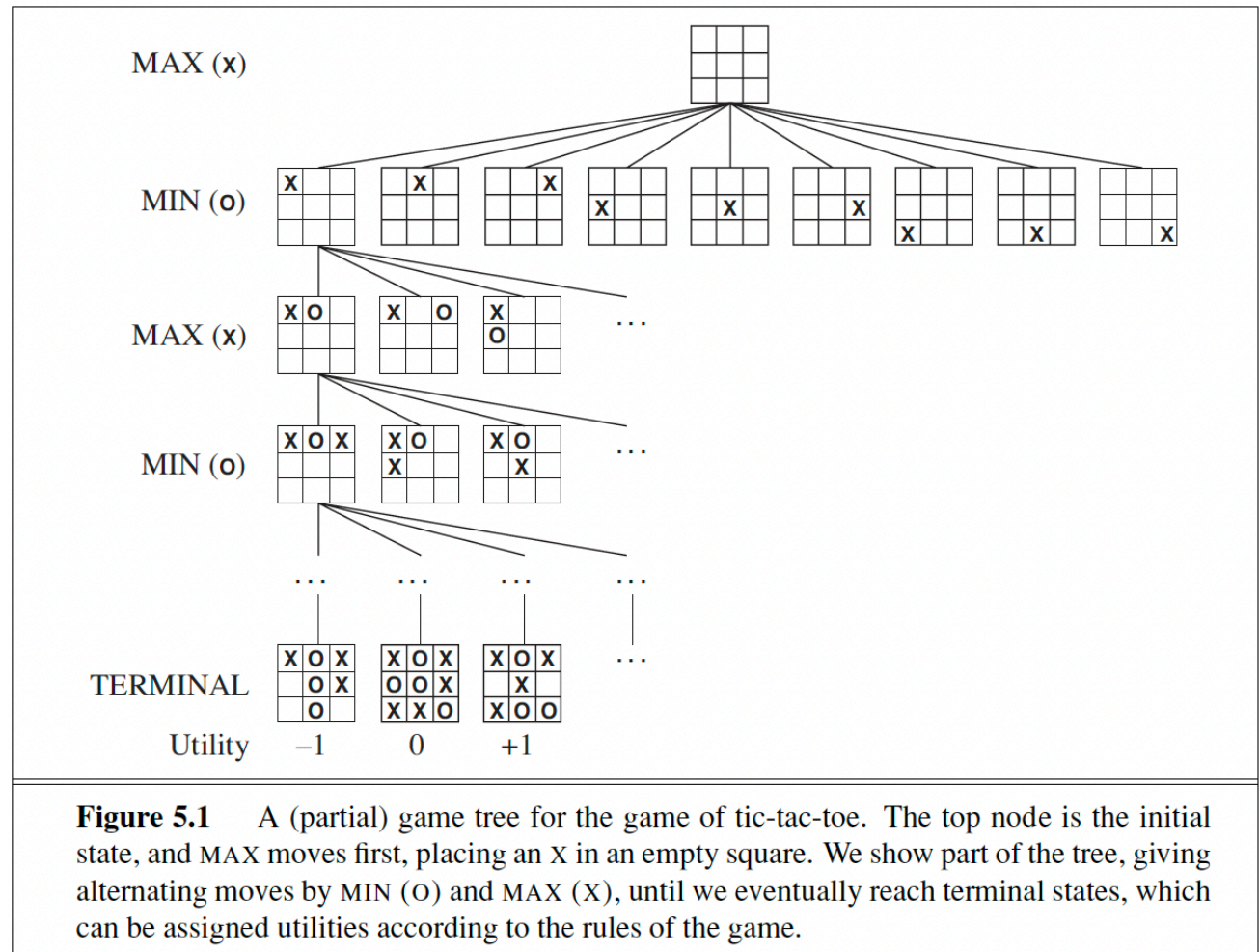
- **S_0** : The **initial state**, which specifies how the game is set up at the start.
- **PLAYER(s)**: Defines **which player has the move** in a state.
- **ACTIONS(s)**: Returns **the set of legal moves** in a state.
- **RESULT(s, a)**: The **transition model**, which **defines the result of a move**.
- **TERMINAL-TEST(s)**: A terminal test, which is **true** when the **game is over** and **false** otherwise. States where the game has ended are called **terminal states**.
- **UTILITY(s, p)**: A utility function (also called an **objective** function or **payoff** function), defines the **final numeric value** for a **game** that **ends in terminal state s** for a **player p**.

Game as a search problem

- A **zero-sum game** is defined as one where the **total payoff** to all players is the **same** for every instance of the game. Chess is zero-sum because every game has payoff of either $0 + 1$, $1 + 0$ or $1/2 + 1/2$.
- The **initial** state, **ACTIONS** function, and **RESULT** function define the **game tree** for the game—a tree where the nodes are game states and the edges are moves.

Tic Tac Toe – Game Tree

- From the initial state, MAX has nine possible moves.
- Play alternates between **MAX's placing an X** and **MIN's placing an O** until we reach **leaf nodes** corresponding to **terminal states** such that **one player has three in a row** or **all the squares are filled**.
- The number on each leaf node indicates the **utility value** of the **terminal state** from the **point of view of MAX**;
 - high values are assumed to be good for MAX and bad for MIN

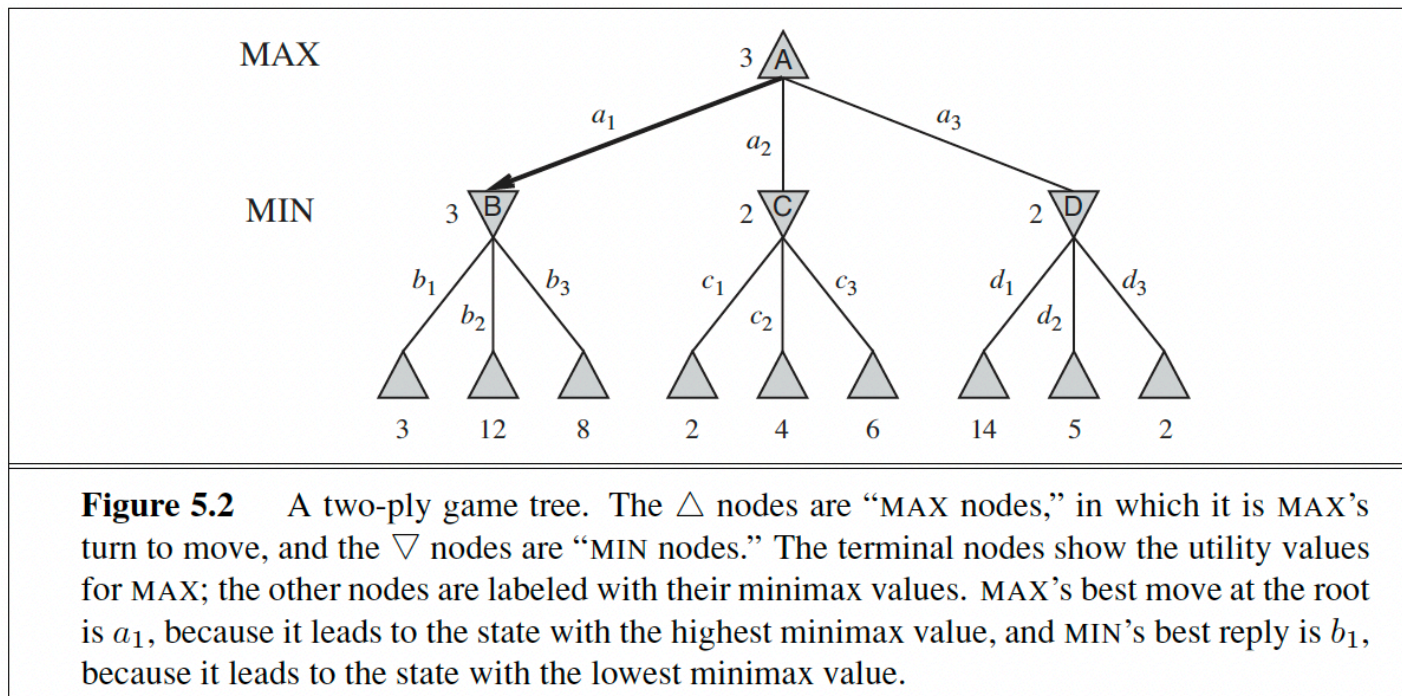


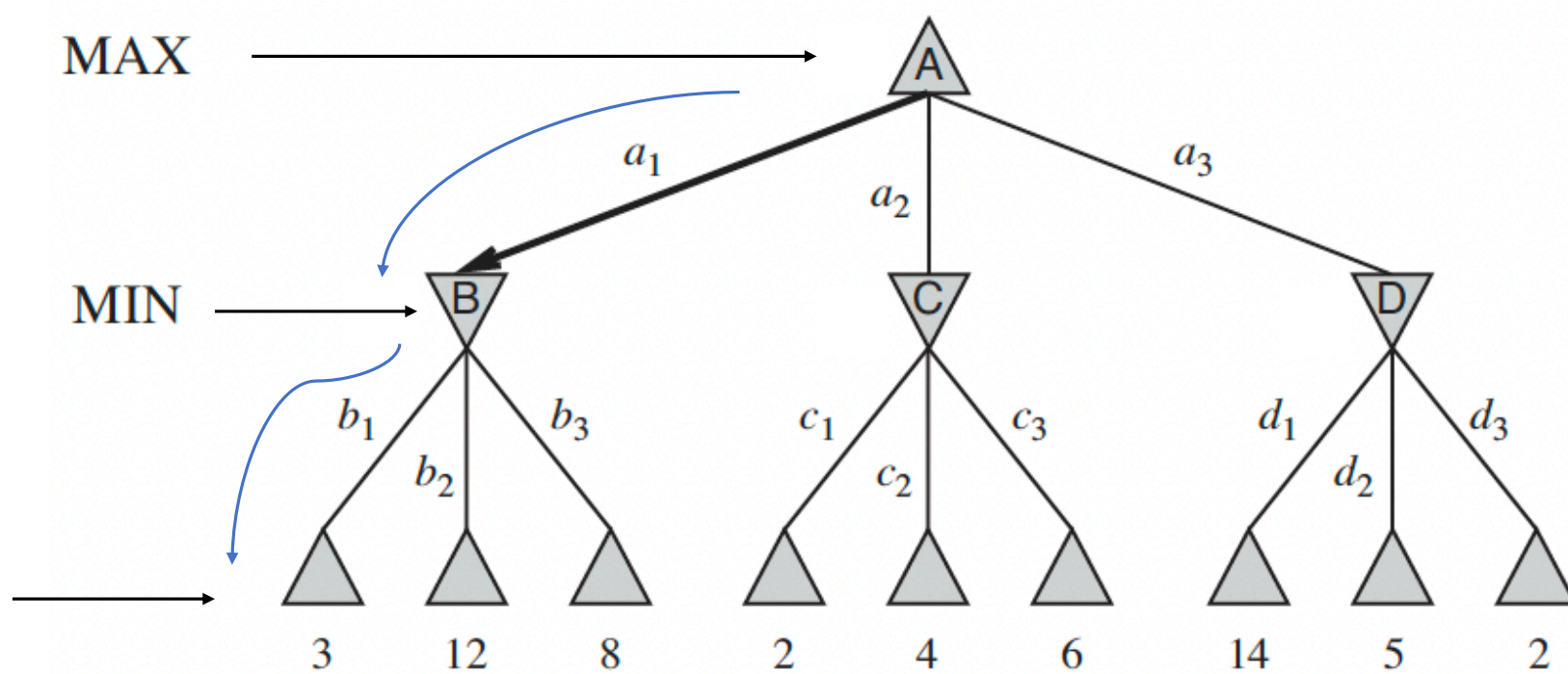
Minimax Value

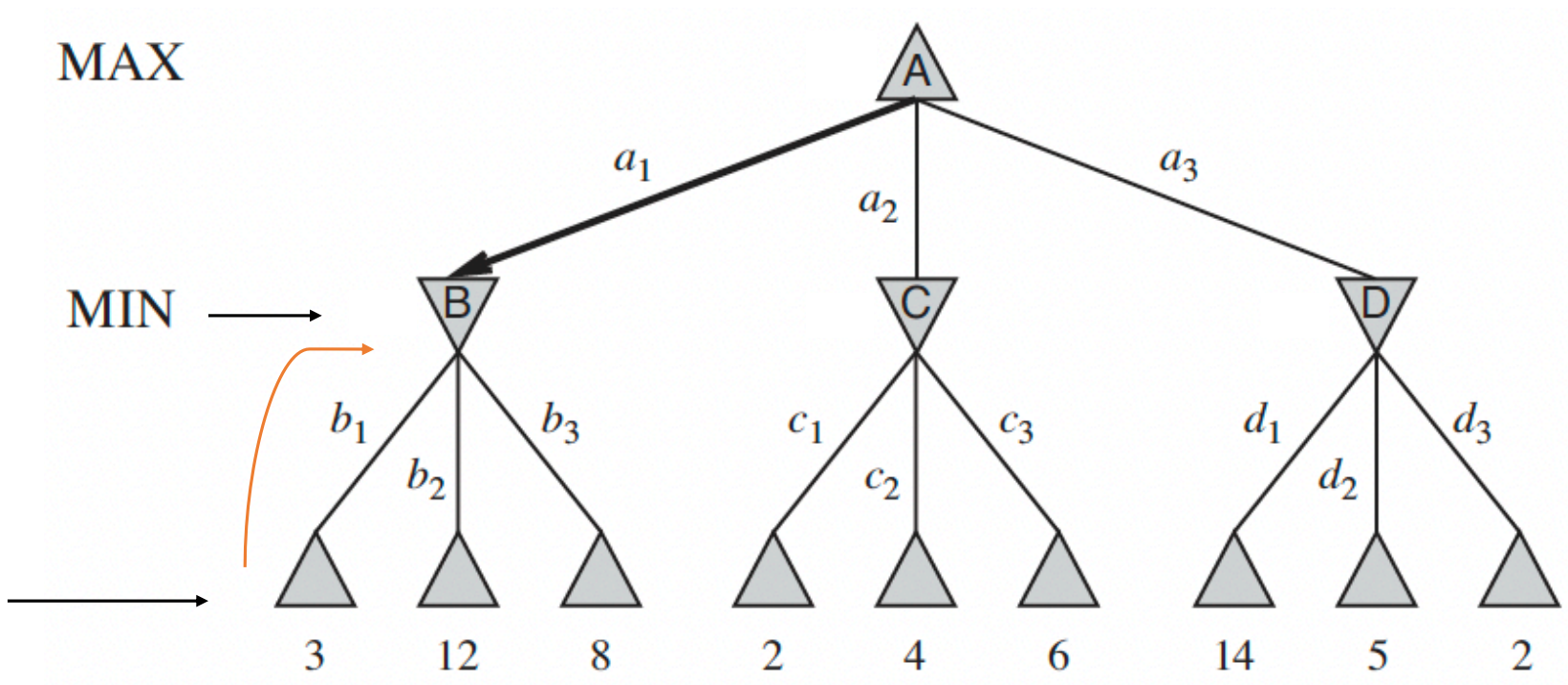
- Given a game tree, the optimal strategy can be determined from the **minimax value of each node**; MINIMAX(n).
- The minimax value of a node is the **utility** (for MAX) of being in the corresponding state, assuming that both players play optimally from there to the end of the game.
- Obviously, the **minimax value of a terminal state** is just its **utility**.
- Furthermore, given a choice, **MAX prefers** to move to a state of **maximum value**, whereas **MIN prefers** a state of **minimum value**.

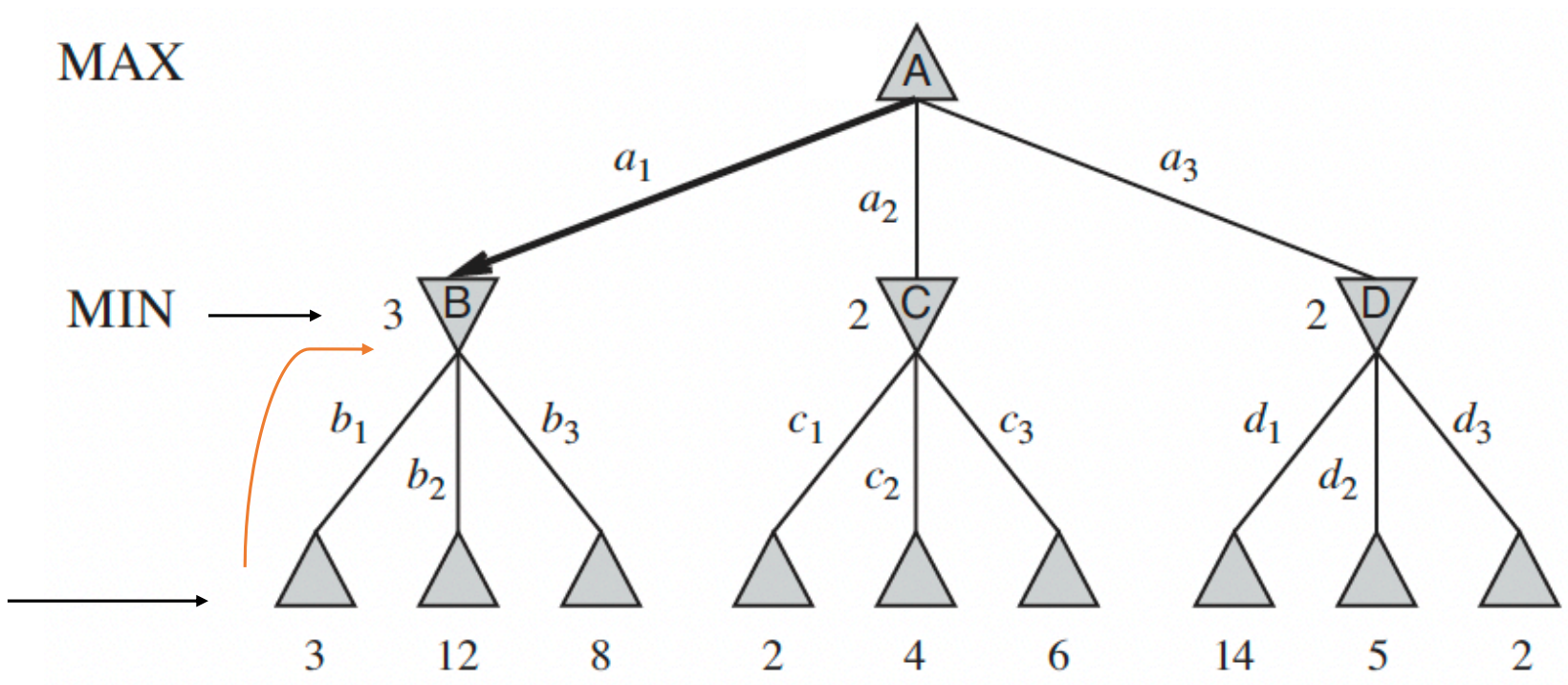
$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

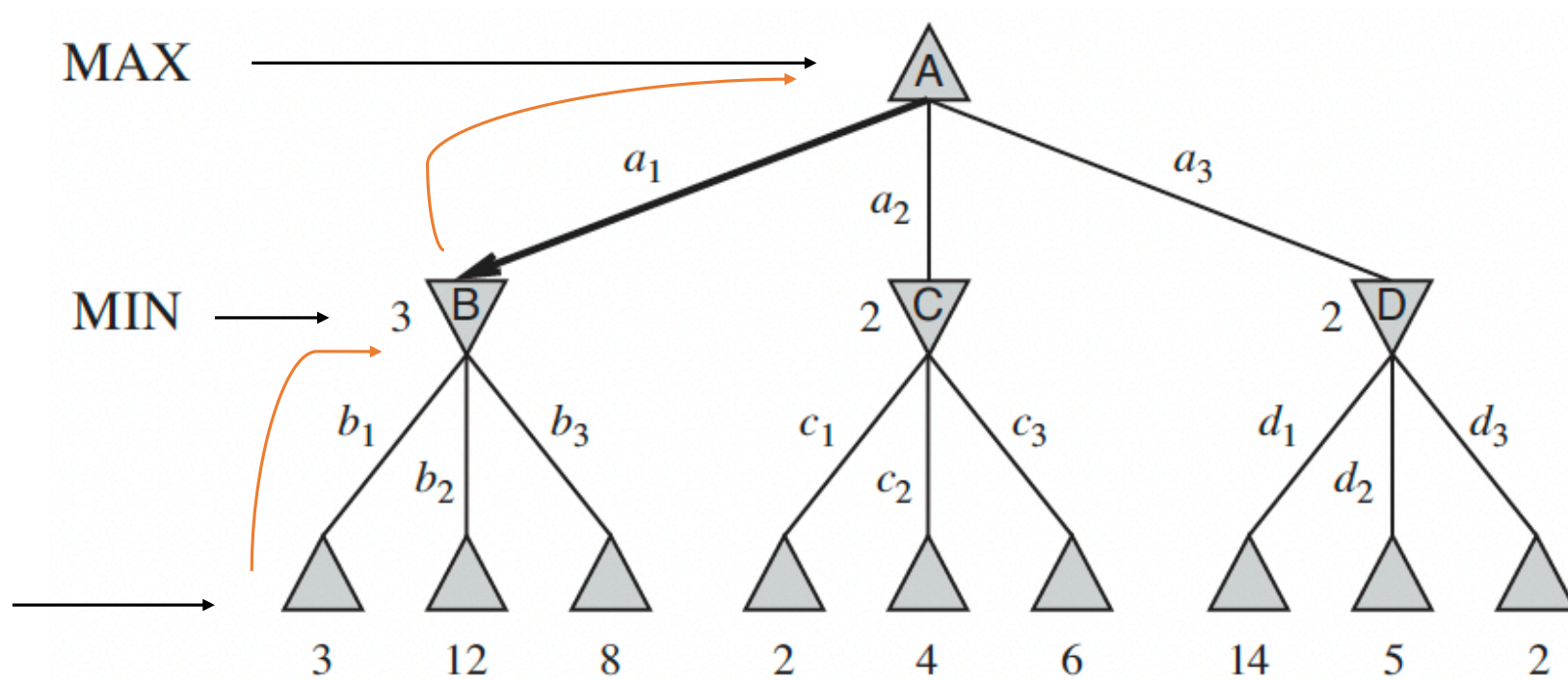
- The **possible moves for MAX** at the root node are labeled a_1 , a_2 , and a_3 .
- The **possible replies to a_1 for MIN** are b_1 , b_2 , b_3 , and so on.
- This particular **game ends after one move each** by MAX and MIN.
- In game parlance, we say that this tree is one move deep, consisting of **two half-moves**, each of which is called a **ply**.
- The **utilities of the terminal states** in this game range from 2 to 14.

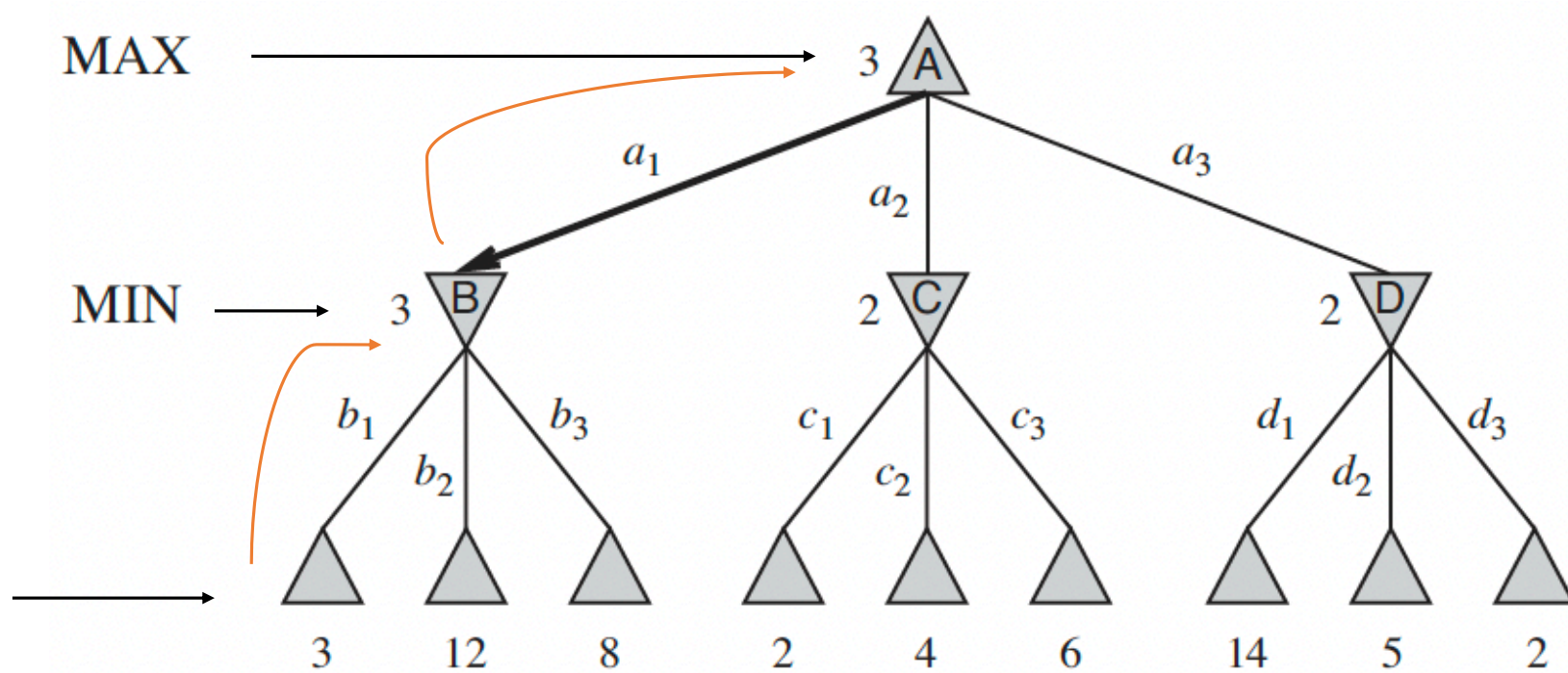










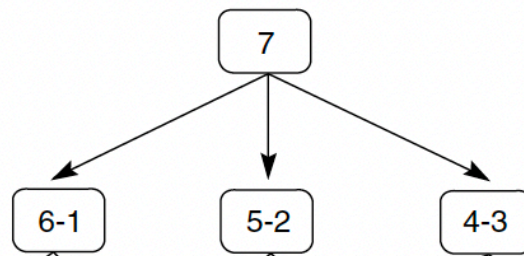


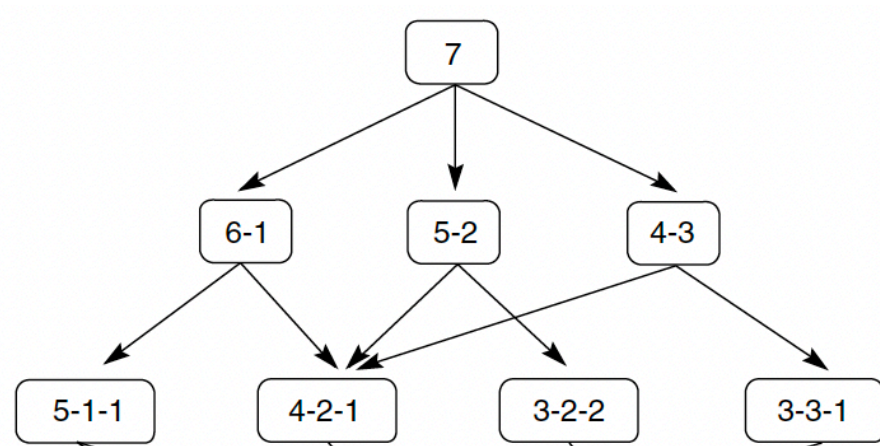
Minimax Algorithm

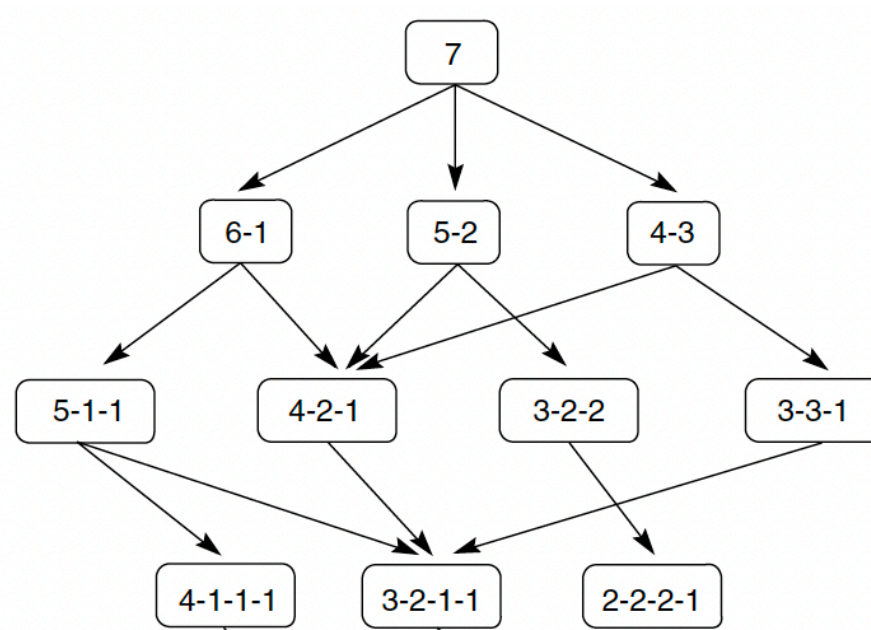
- The minimax algorithm computes the minimax decision from the current state.
- It uses a simple recursive computation of the minimax values of each successor state, directly implementing the defining equations.
- The recursion proceeds all the way down to the leaves of the tree, and then the minimax values are backed up through the tree as the recursion unwinds.

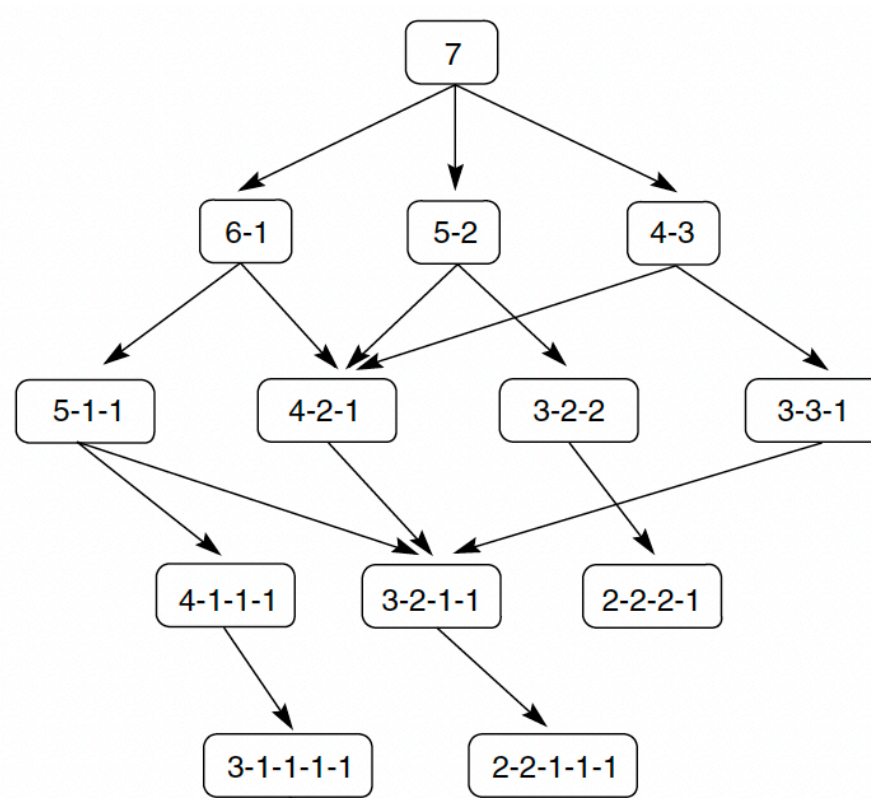
Example - nim

- We first consider a variant of the game **nim**, whose state space may be exhaustively searched.
- To play this game, **a number of tokens** are placed on a table between the **two opponents**;
- At each move, the **player must divide a pile of tokens into two nonempty piles of different sizes**.
- Thus, **6 tokens** may be divided into piles of 5 and 1 or 4 and 2, but not 3 and 3.
- The **first player who can no longer make a move loses** the game.









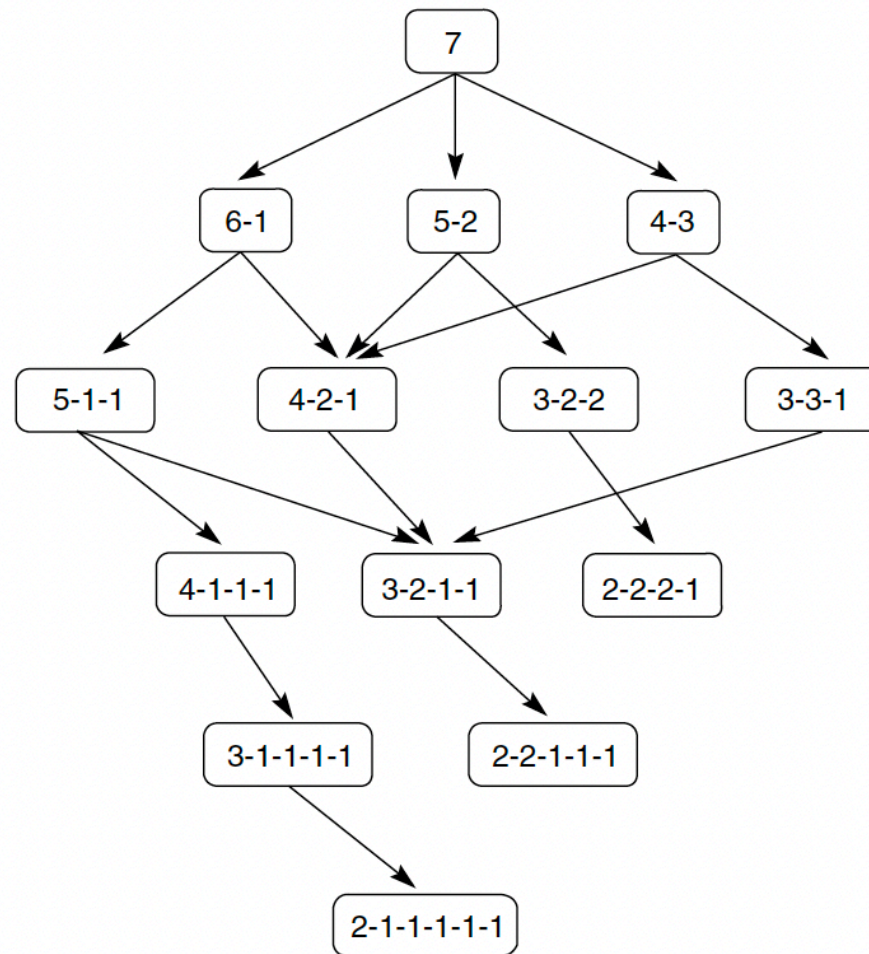


Figure 4.19 State space for a variant of nim. Each state partitions the seven matches into one or more piles.

- The **opponents** in a game are referred to as **MIN** and **MAX**.
- MAX represents the player trying to win, or to **MAXimize** her advantage.
- MIN is the opponent who attempts to **MINimize** MAX's score.
- We assume that MIN uses the same information and always attempts to move to a state that is worst for MAX.
- In implementing minimax, we **label each level in the search space according to whose move** it is at that point in the game, MIN or MAX.
- Each **leaf node** is **given a value of 1 or 0**, depending on whether it is a **win for MAX or for MIN**.
- Minimax propagates these values up the graph through successive parent nodes according to the rule:
 - If the parent state is a MAX node, give it the maximum value among its children.
 - If the parent is a MIN node, give it the minimum value of its children.

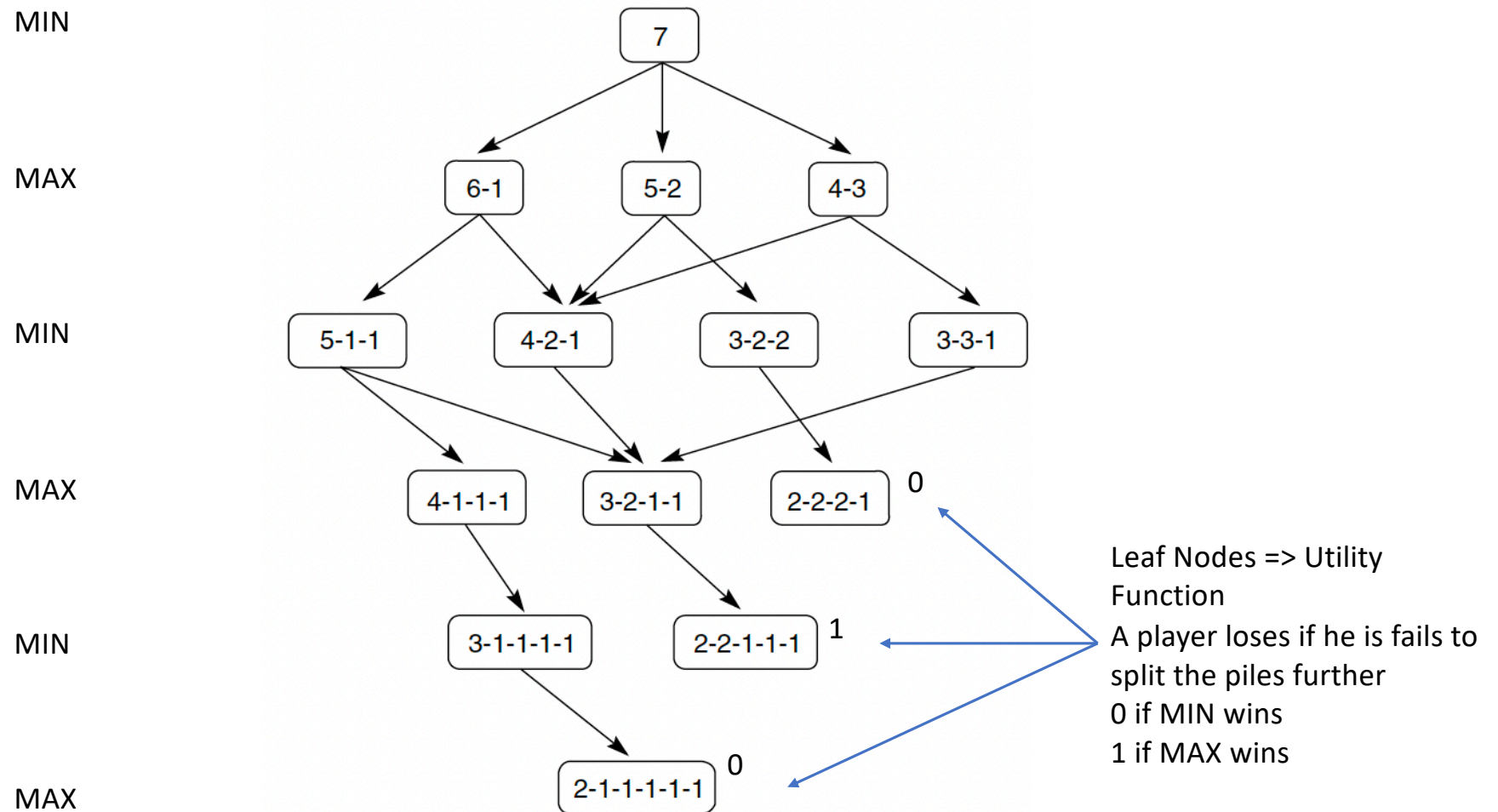


Figure 4.19 State space for a variant of nim. Each state partitions the seven matches into one or more piles.

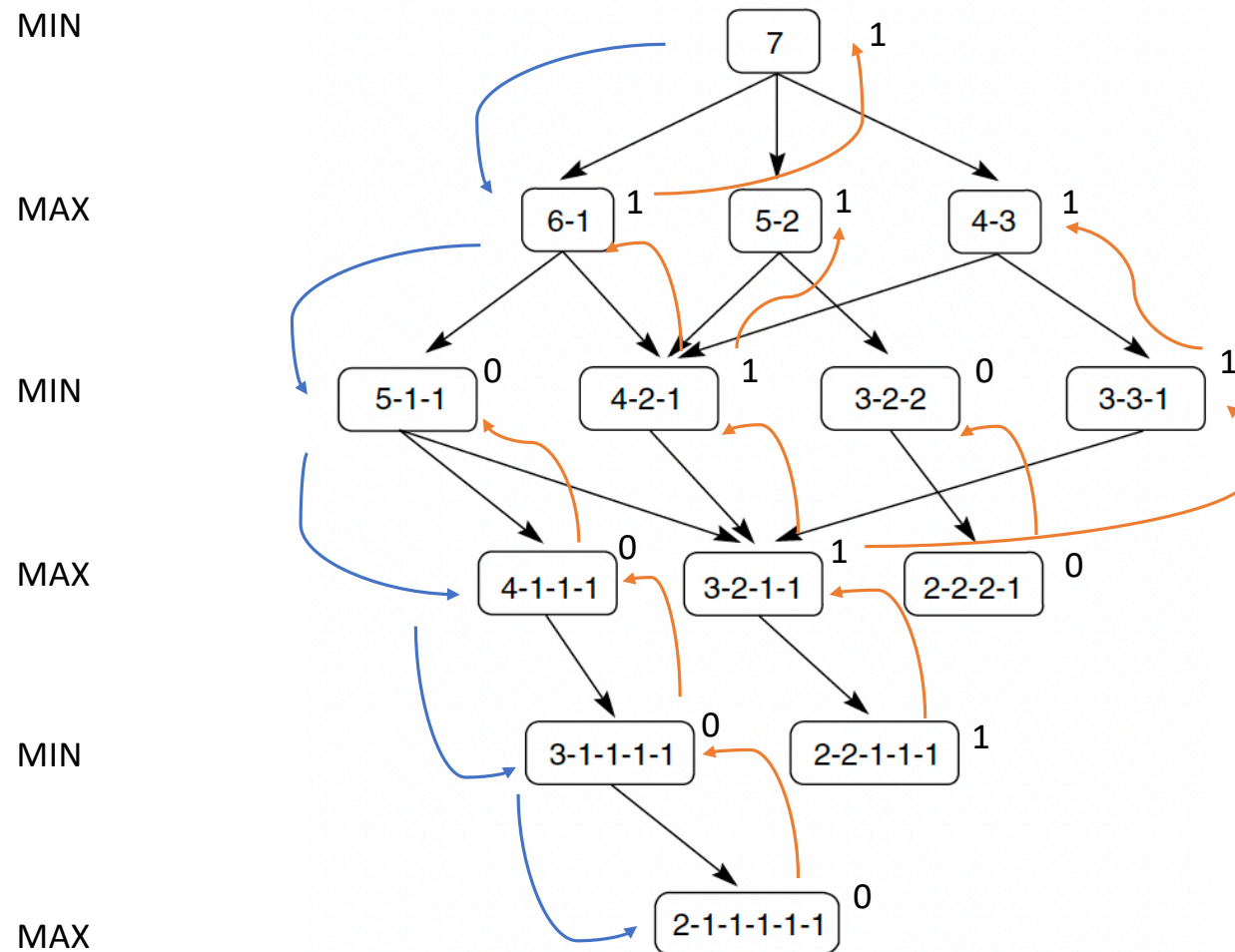


Figure 4.19 State space for a variant of nim. Each state partitions the seven matches into one or more piles.

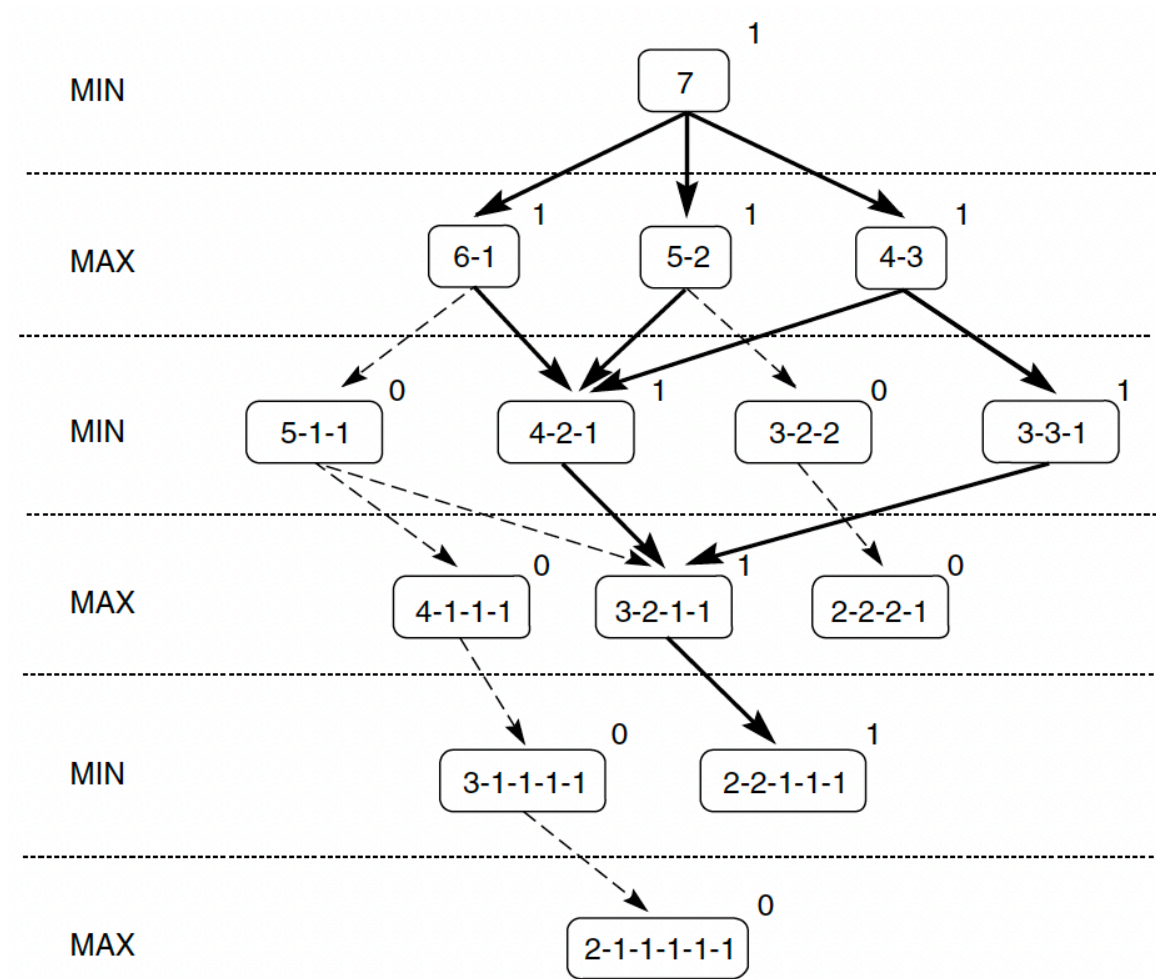


Figure 4.20 Exhaustive minimax for the game of nim. Bold lines indicate forced win for MAX. Each node is marked with its derived value (0 or 1) under minimax.

Activity – Minimax

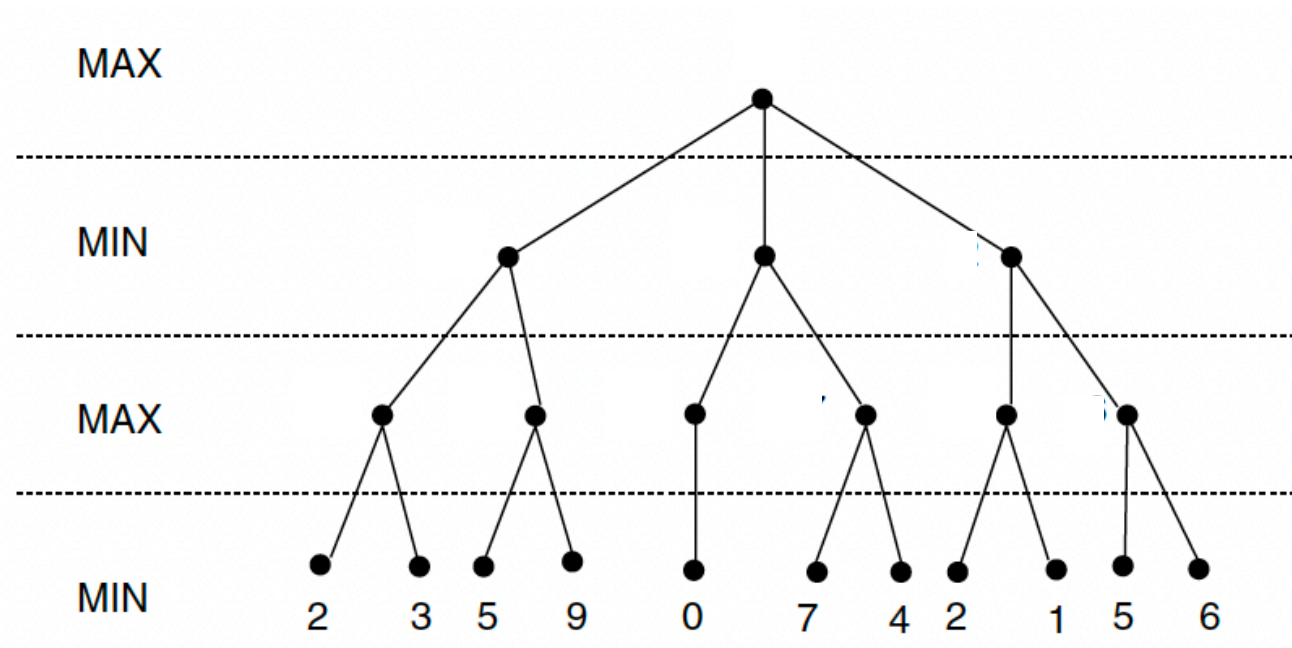


Figure 4.21 Minimax to a hypothetical state space. Leaf states show heuristic values; internal states show backed-up values.

Solution

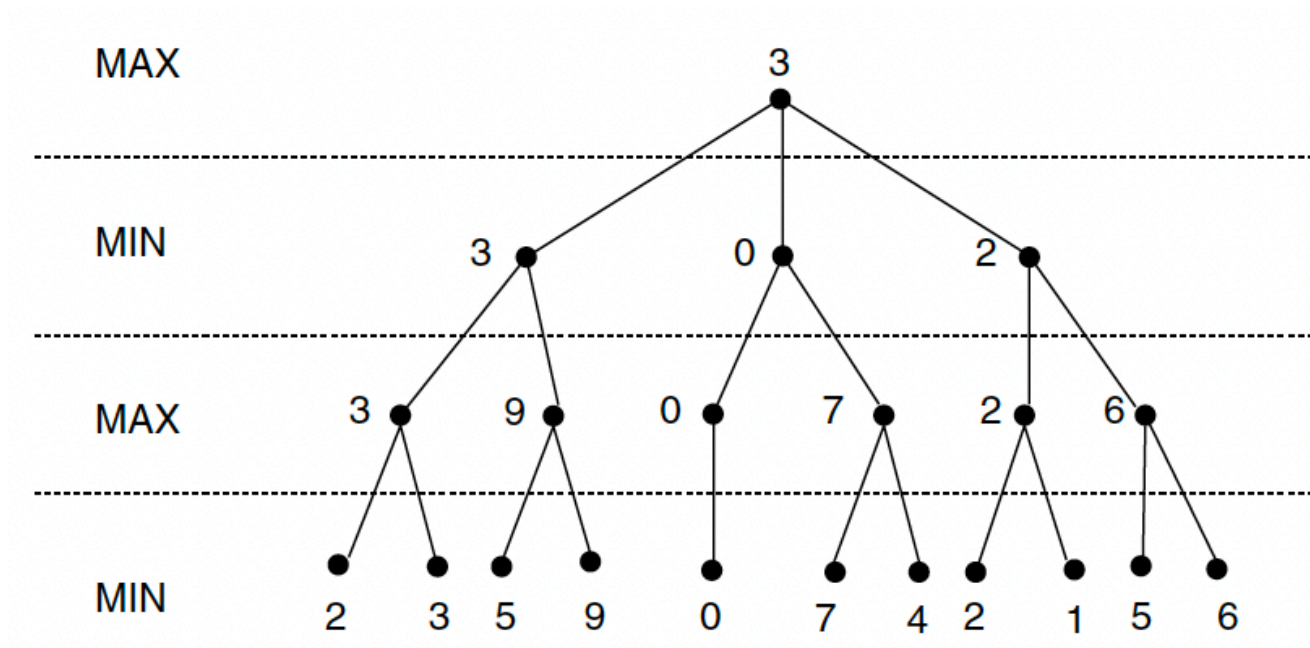


Figure 4.21 Minimax to a hypothetical state space. Leaf states show heuristic values; internal states show backed-up values.